CHAPTER 12

Interior Mutability, Guards and Lifetimes

The borrow checker enforces one rule above all: at any point, a value has either any number of readers or exactly one writer. Most of the time that rule costs you nothing. Sometimes it is in the way. A cache wants to fill itself in a method that takes `&self`. A command history is appended to by an editor and by an autosave task that both hold a shared reference to it. A counter in a struct is bumped by every read. Several threads push results into one vector.

Rust's answer is **interior mutability**: types such as `Cell`, `RefCell`, `OnceCell`, `Mutex` and `RwLock` allow mutation through a shared reference, and keep it safe by other means: a check at run time, a lock, or an API that never hands out a reference that a mutation could invalidate. It is tempting to think that these types switch lifetimes off. They do not. Every reference into a cell is still a borrow with a region, and most of these types hand out **guards**, values that carry a lifetime parameter and run code when they are dropped. That is exactly the kind of value whose lifetime Chapter 5 showed to be subtle.

This chapter is about the place where the compile-time checks of this book meet the run-time checks of interior mutability.

In this chapter you will learn:

- what `RefCell::borrow` returns: a guard `Ref<'_, T>` that is a compile-time borrow of the cell and a run-time borrow of its contents;
- why a method cannot return `&T` from inside a `RefCell` (E0515), and three ways out: `Ref::map`, a closure, an owned copy;
- why a guard in a `match`, a `while let` or an extended `let` lives longer than you expect, and what edition 2024 changed;
- how `Mutex` and `RwLock` guards follow the same rules, and how to keep the locked region short;
- which conflicts are compile errors and which are panics or deadlocks;
- why `Cell` needs no guard, why it is invariant, and why `OnceCell::get_or_init` can return a plain `&T`;
- how `thread_local!` and `LocalKey::with` lend a reference to a closure that cannot leak it.

12.1 Two kinds of checking

`&T` and `&mut T` encode the readers-or-writer rule in types, and the borrow checker verifies it before the program runs. Interior mutability keeps the rule but enforces it in a different way. At the bottom of every such type is `UnsafeCell<T>`, the one type through which the language allows mutation behind a shared reference; each safe wrapper adds its own discipline on top:

Type	Mutates through `&self` with	Keeps aliasing safe by	Hands out
`Cell<T>`	`set`, `replace`, `take`	never lending a reference to the contents	copies and moved values
`RefCell<T>`	`borrow_mut`	counting borrows at run time	`Ref`, `RefMut` guards
`OnceCell<T>`	`get_or_init`, once	never changing the contents after that	`&T`
`Mutex<T>`	`lock`	a lock that blocks other threads	`MutexGuard`
`RwLock<T>`	`write`	a readers-writer lock	read and write guards

The first three are for a single thread; `Mutex` and `RwLock` are their thread-safe counterparts (as `OnceLock` is for `OnceCell`, Chapter 7). The rest of the chapter takes them in the order that matters for lifetimes: first the guards, then the types that need none.

12.2 `RefCell` and its guards

Mutation through `&self`

Here is a command history that can be appended to through a shared reference. The full program is `code/examples/ch12_refcell_guard.rs`:

```
/// A command history that can be appended to through a shared reference.
struct History {
    entries: RefCell<Vec<String>>,
}

impl History {
    fn new() -> Self {
        History {
            entries: RefCell::new(Vec::new()),
        }
    }

    fn record(&self, entry: &str) {
        self.entries.borrow_mut().push(entry.to_owned()); // `&self` is enough
    }

    fn len(&self) -> usize {
        self.entries.borrow().len()
    }
}
```

`record` takes `&self`, yet it pushes to the vector. Two parts of a program can now hold `&History` at the same time and both record entries:

```
let history = History::new();
let editor = &history;
```

```
let autosave = &history; // two shared references to the same history
editor.record("open notes.txt");
autosave.record("save notes.txt");
println!("{} entries", history.len());
```

```
2 entries
```

What `borrow` returns

The signatures explain everything else. Simplified from the standard library's source (Rust 1.99; attributes, `const` and the `?Sized` bounds omitted), they read:

```
impl<T> RefCell<T> {
    pub fn borrow(&self) -> Ref<'_, T>;
    pub fn borrow_mut(&self) -> RefMut<'_, T>;
    pub fn try_borrow_mut(&self) -> Result<RefMut<'_, T>, BorrowMutError>;
}

pub struct Ref<'b, T> { /* a pointer to the value and to the borrow count */ }
```

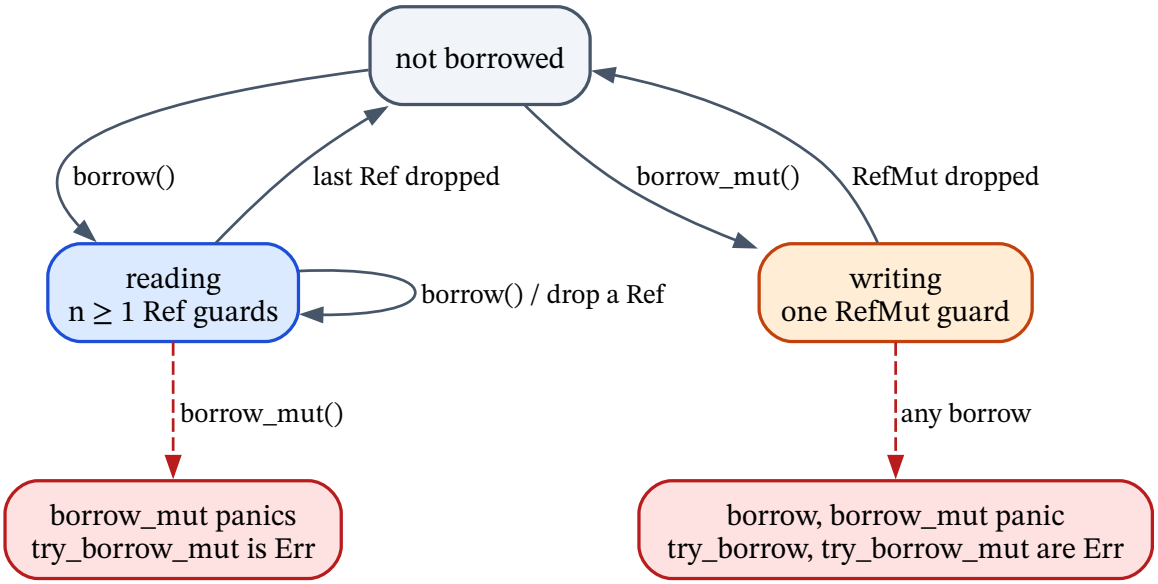
By elision (Chapter 3), the `'_` in the result is the lifetime of `&self`: the guard *borrows the cell*. That gives every guard two layers of checking.

At compile time, a `Ref<'b, T>` holds an ordinary shared borrow of the cell for `'b`. Everything this book has said about borrows applies to it: the cell must outlive the guard, it cannot be moved while a guard exists, and so on. This program, `code/compile_fail/ch12_guard_outlives_cell.rs`, is rejected exactly as if `entries` were a `&Vec<String>`:

```
pub fn entry_count() -> usize {
    let entries;
    {
        let history = RefCell::new(vec![String::from("open")]);
        entries = history.borrow();
    }
    entries.len()
}
```

```
error[E0597]: `history` does not live long enough
  --> compile_fail/ch12_guard_outlives_cell.rs:12:19
   |
11 |         let history = RefCell::new(vec![String::from("open")]);
   |             ------- binding `history` declared here
12 |         entries = history.borrow();
   |                   ^^^^^^^ borrowed value does not live long enough
13 |     }
   |     - `history` dropped here while still borrowed
14 |     entries.len()
   |     ------- borrow later used here
```

At run time, the cell counts its guards. `borrow` succeeds when there is no `RefMut`, and adds one reader; `borrow_mut` succeeds only when there is no guard at all; and each guard's destructor gives its borrow back. The compiler cannot do this part, because `borrow` and `borrow_mut` both take `&self`: to the borrow checker, two calls are two shared borrows of the cell, which never conflict.



The run-time borrow state of a `RefCell`. Guards move the cell between states; a conflicting request panics, or returns an error with the `try_` methods.

`try_borrow_mut` asks for a `RefMut` without panicking, which makes the run-time state visible:

```
let entries = history.entries.borrow(); // a `Ref`: a shared borrow at run time
match history.entries.try_borrow_mut() {
    Ok(_) => println!("with a Ref alive: got a RefMut"),
    Err(error) => println!("with a Ref alive: {error}"),
}
println!("still readable: {:?}", entries.first());
drop(entries); // the run-time borrow ends here, not at the closing brace
match history.entries.try_borrow_mut() {
    Ok(mut entries) => {
        entries.push(String::from("close notes.txt"));
        println!("after drop: got a RefMut");
    }
    Err(error) => println!("after drop: {error}"),
}
```

```
with a Ref alive: RefCell already borrowed
still readable: Some("open notes.txt")
after drop: got a RefMut
```

The `Ref` named `entries` keeps the cell in the "reading" state until `drop(entries)`; the next request succeeds. For a guard, the end of its *region* in the borrow checker's sense (the last use) and the end of the run-time borrow (the drop) are different points. A guard that is no longer used but not yet dropped still blocks writers. With guards, *when a value is dropped* matters far more than *when it was last used*.

Compile error or panic?

Since the run-time layer is invisible to the compiler, a conflict between two guards compiles and panics when it happens. `code/examples/ch12_double_borrow.rs` provokes both conflicts and catches the panics to print their messages:

```
let message = panic_message(|| {
    let reading = words.borrow();
    words.borrow_mut().push(String::from("beta")); // compiles; panics
    println!("never printed: {}", reading.len());
});
println!("borrow_mut while a Ref is alive: {message}");

let message = panic_message(|| {
    let mut writing = words.borrow_mut();
    writing.push(format!("{} words", words.borrow().len())); // panics
});
println!("borrow while a RefMut is alive: {message}");
```

```
borrow_mut while a Ref is alive: RefCell already borrowed
borrow while a RefMut is alive: RefCell already mutably borrowed
after the panics: ["alpha", "gamma"]
```

"RefCell already borrowed" means that `borrow_mut` found a guard; "RefCell already mutably borrowed" means that `borrow` found a `RefMut`. The last line shows that unwinding dropped the guards, so the cell is usable again afterwards.

The rule of thumb is simple. A conflict between two *guards* of the same cell is found at run time. A conflict between a guard and something that needs the cell *itself* (moving it, dropping it, calling a `&mut self` method such as `RefCell::get_mut`) is found at compile time, because the guard is an ordinary borrow of the cell.

Returning a reference from inside a `RefCell` (E0515)

The most common surprise with `RefCell` is a getter that will not compile. A method on `History` that returns the last entry (`code/compile_fail/ch12_return_ref_from_guard.rs`):

```
impl History {
    pub fn last(&self) -> Option<&str> {
        self.entries.borrow().last().map(|entry| entry.as_str())
    }
}
```

```
error[E0515]: cannot return value referencing temporary value
  --> compile_fail/ch12_return_ref_from_guard.rs:14:9
   |
14 |         self.entries.borrow().last().map(|entry| entry.as_str())
   |         ---------------------^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
   |         |
   |         returns a value referencing data owned by the current function
   |         temporary value created here
```

"temporary value created here" points at `self.entries.borrow()`: the guard is a temporary of the method's body. The `&str` is obtained *through* it, by `Deref::deref(&self) -> &Self::Target`, so its lifetime is that of a borrow of the guard, a value that dies when the method returns. Hence "returns a value referencing data owned by the current function", the E0515 of Chapter 5.

It helps to see why this must be so. If you could get a `&'b str` with the lifetime of the guard's *type* instead, you could drop the guard (resetting the count), keep the `&str`, and then call `borrow_mut` and clear the vector, leaving the `&str` dangling. The run-time count only works if every reference into the cell is tied to a live guard, and the signature of `deref` makes sure it is.

Three ways out

There are three honest signatures for "give the caller access to data inside a `RefCell`". `code/examples/ch12_ref_map.rs` implements all three. The first returns a guard, narrowed to the part the caller asked for:

```
impl History {
    /// The whole list, as a guard that derefs to a slice.
    fn entries(&self) -> Ref<'_, [String]> {
        Ref::map(self.entries.borrow(), |entries| entries.as_slice())
    }

    /// The last entry, if any: `filter_map` gives the guard back on `None`.
    fn last(&self) -> Option<Ref<'_, str>> {
        Ref::filter_map(self.entries.borrow(), |entries| {
            entries.last().map(|entry| entry.as_str())
        })
        .ok()
    }
}
```

`Ref::map` takes a guard and a function that projects the borrowed value to a part of it, and returns a guard for that part; the run-time borrow is transferred, not released. `Ref::filter_map` does the same for a projection that may fail, and gives the original guard back in the `Err` case (stable since Rust 1.63). Both are associated functions, called as `Ref::map(guard, f)` rather than `guard.map(f)`: as the standard library's documentation puts it, "A method would interfere with methods of the same name on the contents of a `RefCell` used through `Deref`."

The return type `Option<Ref<'_, str>>` is honest: it says that the caller holds a run-time borrow of the history until it drops the result. The second design does not let the caller hold anything; it lends the data to a closure. The third gives up on borrowing and returns an owned copy:

```
impl History {
    /// Lends the last entry to `f`; the reference cannot escape the call.
    fn with_last<R>(&self, f: impl FnOnce(Option<&str>) -> R) -> R {
        let entries = self.entries.borrow();
        f(entries.last().map(|entry| entry.as_str()))
    }

    /// Or give up on borrowing: an owned copy has no guard at all.
    fn last_owned(&self) -> Option<String> {
        self.entries.borrow().last().cloned()
    }
}
```

In `with_last`, the closure's parameter is `Option<&str>` with an elided lifetime, so the bound is higher-ranked (Chapter 9): `f` must work for a borrow of *any* lifetime, including one that ends when `with_last` returns. Its result type `R` is chosen by the caller before the call and cannot mention that lifetime, so the `&str` cannot escape. We will meet the same shape again with `thread_local!`. Here is how the three look from the caller's side:

```
if let Some(last) = history.last() {
    println!("last: {}", &*last); // `last` is a `Ref<'_, str>`
} // the guard is dropped here, so recording below is fine
history.entries.borrow_mut().push(String::from("save"));

println!("all: {:?}", &*history.entries());
let shouting = history.with_last(|last| last.map(str::to_uppercase));
```

```
println!("shouting: {shouting:?}");
let owned = history.last_owned();
history.entries.borrow_mut().clear(); // no guard is alive
println!("owned: {owned:?}, left: {}", history.entries().len());
```

```
last: edit
all: ["open", "edit", "save"]
shouting: Some("SAVE")
owned: Some("save"), left: 0
```

The comment after the `if let` is the important one: the guard returned by `last()` is released at the end of the `if let`, so the `borrow_mut` on the next line succeeds. Had the caller kept `last` in a variable until the end of the function, that `borrow_mut` would have panicked. A guard in a return type moves the responsibility for the run-time borrow to the caller; a closure or an owned value keeps it inside the method.

12.3 Guards and temporaries

A guard you create inside an expression and do not bind to a variable is a temporary, and the temporary scope rules of Chapter 5 decide when it is dropped. For most temporaries that timing is invisible. For a guard it decides whether the next `borrow_mut` succeeds. `code/examples/ch12_guard_temporaries.rs` uses a small helper to report the state of a cell without panicking:

```
/// Reports whether the cell is borrowed right now, without panicking.
fn status(words: &RefCell<Vec<String>>) -> &'static str {
    if words.try_borrow_mut().is_ok() {
        "free"
    } else {
        "borrowed"
    }
}
```

`match`: alive for every arm

```
match words.borrow().len() {
    0 => println!("match: empty"),
    n => println!("match: {n} words, cell is {}", status(&words)),
}
```

```
match: 2 words, cell is borrowed
```

The scrutinee is `words.borrow().len()`, a plain `usize`, yet the arm runs with the cell borrowed: the guard is a temporary of the scrutinee, and those live until the end of the `match`, in every edition. Call `words.borrow_mut()` in that arm and the program panics. The fix is to compute the value first, with a `let`:

```
let count = words.borrow().len(); // the guard dies at this `;`
println!("after let: {count} words, cell is {}", status(&words));
```

```
after let: 2 words, cell is free
```

`if let`: released before `else` (edition 2024)

```
if let Some(first) = words.borrow().first().cloned() {
    println!("if let, then-branch: {first}, cell is {}", status(&words));
}
if let Some(tenth) = words.borrow().get(9).cloned() {
    println!("if let, then-branch: {tenth}");
} else {
    println!("if let, else-branch: cell is {}", status(&words));
}
```

```
if let, then-branch: alpha, cell is borrowed
if let, else-branch: cell is free
```

The guard lives through the then-branch, and is released before the else-branch. That second half is new in edition 2024. In edition 2021 the guard lived until the end of the whole `if let`, and a common pattern, "look the key up; if it is missing, insert it in the `else` branch", panicked with "RefCell already borrowed". Chapter 5 shows the edition 2021 rule with a program that the compiler rejects (`ch05_if_let_2021.rs`); with a `RefCell`, the same mistake compiles and fails at run time, which is worse.

Extended `let`: alive until the end of the block

```
{
    let first = &words.borrow()[0]; // the guard is lifetime-extended
    println!("extended: {first}, cell is {}", status(&words));
}
```

```
extended: alpha, cell is borrowed
```

This one is easy to miss. `&words.borrow()[0]` borrows an element of the vector *through* the guard, and the borrow appears in an extending position of the `let` (the operand of `&`, Chapter 5). So the guard is lifetime-extended to the end of the block, as if you had written `let guard = words.borrow();`. Without the extension, `first` would point into a guard that was already dropped. Either way, a reference into a `RefCell` keeps a guard alive, visibly or not.

`while let`: alive for each iteration

```
while let Some(word) = words.borrow_mut().pop() {
    println!("while let body: {word}, cell is {}", status(&words));
}
```

```
while let body: beta, cell is borrowed
while let body: alpha, cell is borrowed
```

The `RefMut` created by `words.borrow_mut()` lives through the whole body of each iteration, not only through the `pop()`. A work queue whose loop body pushes follow-up jobs (`words.borrow_mut().push(...)`) panics on the first iteration. The fix has the same shape as for `match`: pop in a statement of its own, then look at the result (we will write it out for `Mutex` below).

Tail expressions (edition 2024)

The second edition 2024 change of Chapter 5 matters for guards too. A function whose last expression reads a local `RefCell` creates the guard in its tail expression. In edition 2021, that guard was dropped after the local it borrows, and the function did not compile (`code/compile_fail/ch12_tail_guard_2021.rs`, compiled with `edition=2021`):

```
pub fn entry_count() -> usize {
    let history = RefCell::new(vec![String::from("open")]);
    history.borrow().len()
}
```

```
error[E0597]: `history` does not live long enough
  --> compile_fail/ch12_tail_guard_2021.rs:11:5
   |
10 |     let history = RefCell::new(vec![String::from("open")]);
   |         ------- binding `history` declared here
11 |     history.borrow().len()
   |     ^^^^^^^----------
   |     |
   |     borrowed value does not live long enough
   |     a temporary with access to the borrow is created here ...
12 | }
   | -
   | |
   | `history` dropped here while still borrowed
   | ... and the borrow might be used here, when that temporary is dropped and runs the
destructor for type `Ref<'_, Vec<String>>`
   |
   = note: the temporary is part of an expression at the end of a block;
           consider forcing this temporary to be dropped sooner, before the block's local
variables are dropped
help: for example, you could save the expression's value in a new local variable `x` and
then make `x` be the expression at the end of the block
   |
11 |     let x = history.borrow().len(); x
   |     +++++++                       +++
```

The story is the one Chapter 5 read label by label: a temporary with access to the borrow is created in the tail expression, `history` is dropped at the closing brace, and then the borrow "might be used here, when that temporary is dropped and runs the destructor for type `Ref<'_, Vec<String>>`". (Chapter 5's `Reader` had "the `Drop` code for type"; `Ref` has no `Drop` impl of its own, but one of its fields has, so the compiler speaks of its *destructor*.) In edition 2024 the guard is dropped before the locals, and the function compiles unchanged: it is `entry_count` in `ch12_guard_temporaries.rs`, whose last line of output is `entry_count() = 1`.

Here is the whole section in one table:

You write	The guard is released
`let n = cell.borrow().len();`	at the `;`
`let guard = cell.borrow();`	when `guard` goes out of scope, or at `drop(guard)`
`let first = &cell.borrow()[0];`	at the end of the block (extended)
`match cell.borrow().len() { .. }`	after the `match`
`if let .. = cell.borrow().. { } else { }`	before `else` (2024), after it (2021)
`while let .. = cell.borrow_mut().pop() { }`	at the end of each iteration
`cell.borrow().len()` as a tail expression	before the locals (2024), after them (2021)

12.4 `Mutex`, `RwLock` and their guards

A `MutexGuard` is a borrow of the mutex

`Mutex<T>` is the thread-safe relative of `RefCell<T>`, and its signature has the same shape (simplified the same way):

```
impl<T> Mutex<T> {
    pub fn lock(&self) -> LockResult<MutexGuard<'_, T>>;
    pub fn try_lock(&self) -> TryLockResult<MutexGuard<'_, T>>;
}
```

The guard borrows the mutex for `'_`, gives access to the `T` through `Deref` and `DerefMut`, and unlocks the mutex in its destructor. (`LockResult` is a `Result` because of *poisoning*: if a thread panics while holding the guard, later `lock` calls return an error that still contains the guard. `unwrap()` is the usual response.) Everything in the previous two sections applies with one difference: a conflict does not panic, it *waits*, because another thread may release the lock. On the same thread, nobody will. The documentation of `Mutex::lock` says: "The exact behavior on locking a mutex in the thread which already holds the lock is left unspecified. However, this function will not return on the second call (it might panic or deadlock, for example)."

So the examples below use `try_lock`, which returns `Err(TryLockError::WouldBlock)` instead of waiting (`code/examples/ch12_mutex_guards.rs`):

```
/// Reports whether the mutex is locked right now, without blocking.
fn status<T>(mutex: &Mutex<T>) -> &'static str {
    match mutex.try_lock() {
        Ok(_) => "free",
        Err(TryLockError::WouldBlock) => "locked",
        Err(TryLockError::Poisoned(_)) => "poisoned",
    }
}
```

Sharing a mutex between scoped threads

With `thread::scope` (Chapter 7), threads can borrow the mutex directly: the scope guarantees that they finish before `lengths` is dropped, so no `Arc` is needed:

```
let lengths = Mutex::new(Vec::new());
thread::scope(|s| {
```

```
    for word in ["alpha", "beta", "gamma"] {
        let lengths = &lengths; // a plain shared borrow: no `Arc` needed
        s.spawn(move || {
            let len = word.len(); // work without holding the lock
            lengths.lock().unwrap().push((word, len)); // lock, push, unlock
        });
    }
});
let mut lengths = lengths.into_inner().unwrap();
lengths.sort();
println!("{lengths:?}");
```

```
[("alpha", 5), ("beta", 4), ("gamma", 5)]
```

Each thread does its work (here, `word.len()`) *before* taking the lock, and the guard is a temporary of the statement `lengths.lock().unwrap().push(...)`, so the lock is held for the push and released at the `;`. Keeping the locked region short is the main design rule for mutexes, and the temporary scope rules are how you control it.

Scoping a guard

When you need the lock for several statements, bind the guard and give it a block:

```
let queue = Mutex::new(vec!["parse", "check"]);
{
    let mut jobs = queue.lock().unwrap();
    jobs.push("emit");
    println!("inside the block: {}", status(&queue));
} // `jobs` is dropped: unlocked
println!("after the block: {}", status(&queue));
```

```
inside the block: locked
after the block: free
```

`drop(jobs)` would release it just as well, earlier than the end of the block. What does *not* work is `let _ = queue.lock().unwrap();` to "hold the lock for the rest of the function": `_` binds nothing, so the guard is dropped at the end of the statement (Chapter 5). The mistake is common enough that rustc has a deny-by-default lint for it (`code/compile_fail/ch12_let_underscore_lock.rs`):

```
pub fn append(queue: &Mutex<Vec<String>>, job: &str) {
    let _ = queue.lock().unwrap(); // meant to hold the lock
    println!("appending {job}");
}
```

```
error: non-binding let on a synchronization lock
  --> compile_fail/ch12_let_underscore_lock.rs:10:9
   |
10 |     let _ = queue.lock().unwrap(); // meant to hold the lock
   |         ^ this lock is not assigned to a binding and is immediately dropped
   |
   = note: `#[deny(let_underscore_lock)]` (part of `#[deny(let_underscore)]`) on by default
help: consider binding to an unused variable to avoid immediately dropping the value
   |
10 |     let _unused = queue.lock().unwrap(); // meant to hold the lock
   |          ++++++
help: consider immediately dropping the value
   |
10 -     let _ = queue.lock().unwrap(); // meant to hold the lock
```

```
10 +     drop(queue.lock().unwrap()); // meant to hold the lock
   |
```

The first help is the fix you meant: bind the guard to a name that starts with an underscore, such as `_guard`, which *is* a binding and keeps the guard until the end of the scope.

The `while let` trap with a mutex

The `while let` rule from the previous section turns into a deadlock with a mutex:

```
while let Some(job) = queue.lock().unwrap().pop() {
    // Still locked here: calling `queue.lock()` would never return.
    println!("while let, job {job}: {}", status(&queue));
}
```

```
while let, job emit: locked
while let, job check: locked
while let, job parse: locked
```

The body runs with the queue locked. If it called `queue.lock()` to push a follow-up job, the thread would wait for itself forever (or panic: the behavior is unspecified). The fix is to take the job out in a statement of its own, so that the guard dies at the `;` before the body runs:

```
loop {
    let next = queue.lock().unwrap().pop(); // the guard dies at this `;`
    let Some(job) = next else { break };
    println!("loop, job {job}: {}", status(&queue));
}
```

```
loop, job check: free
loop, job parse: free
```

The same applies to `match queue.lock().unwrap().pop() { .. }` in every edition, and to `if let` in edition 2021.

> **WARNING**
>
> A `RefCell` conflict panics, which you will notice in your tests. A `Mutex` conflict on the same thread may deadlock, and a lock held longer than needed only makes other threads wait, which you may not notice at all. Review every `match`, `while let` and extended `let` whose scrutinee or initializer calls `lock()`.

Readers and writers: `RwLock`

`RwLock<T>` allows many readers or one writer, with `read()` and `write()` returning `RwLockReadGuard<'_, T>` and `RwLockWriteGuard<'_, T>`. The rules for the guards are the same (`code/examples/ch12_rwlock_static.rs`):

```
fn rwlock_demo() {
    let config = RwLock::new(String::from("verbose=false"));
    {
        let first = config.read().unwrap();
        let second = config.read().unwrap(); // many readers at once
        let can_write = config.try_write().is_ok(); // a writer must wait
        println!("readers: {first} and {second}; can write: {can_write}");
```

```
    } // both read guards dropped
    config.write().unwrap().replace_range(8.., "true");
    println!("after writing: {}", config.read().unwrap());
}
```

```
readers: verbose=false and verbose=false; can write: false
after writing: verbose=true
```

Two read guards coexist, and a writer must wait until both are dropped. The write happens through a temporary guard in a statement of its own, and the final read guard is a temporary of the `println!` statement.

Guards of a `static`

The lifetime in a guard's type is the lifetime of the borrow of the lock. When the lock is a `static`, the borrow can be `'static`, and so can the guard:

```
static LOG: Mutex<Vec<String>> = Mutex::new(Vec::new());

fn log(line: &str) {
    LOG.lock().unwrap().push(line.to_owned());
}

/// The mutex is a `static`, so its guard can be `MutexGuard<'static, _>` and
/// can be returned from a function without any lifetime parameter.
fn log_entries() -> MutexGuard<'static, Vec<String>> {
    LOG.lock().unwrap()
}
```

```
log: ["started", "loaded config"]
```

`log_entries` returns a `MutexGuard<'static, Vec<String>>` with no lifetime parameter in sight, because `LOG` lives for the whole program. Note what `'static` does *not* mean here: the lock is held exactly as long as the guard lives, which should still be short. `'static` describes the *mutex*, not the critical section.

One tool you might look for is missing on stable Rust: there is no `MutexGuard::map` to narrow a guard the way `Ref::map` does. In Rust 1.99, `MappedMutexGuard` and the `map` functions of the lock guards are unstable (feature `mapped_lock_guards`, tracking issue rust-lang/rust#117108). For a `Mutex`, use the closure or the owned-copy design from the `RefCell` section, or return the whole guard.

12.5 `Cell`, `OnceCell` and invariance

`Cell`: no references, no guards

`Cell<T>` takes the opposite approach to `RefCell`. It never lends a reference to its contents through `&self`: you can copy the value out (`get`, for `T: Copy`), put a value in (`set`), swap one in (`replace`) or take it out (`take`, for `T: Default`). Since no reference into a `Cell` can exist while someone holds only `&Cell<T>`, a `set` can never invalidate one, and nothing needs to be counted. Here is a document that counts how often its summary is read (`code/examples/ch12_cell_oncecell.rs`):

```
struct Document {
    text: String,
    reads: Cell<u32>,          // a statistic updated through `&self`
    summary: OnceCell<String>, // computed on first use, then fixed
}

impl Document {
    fn new(text: &str) -> Self {
        Document {
            text: text.to_owned(),
            reads: Cell::new(0),
            summary: OnceCell::new(),
        }
    }

    /// The returned `&str` borrows `self`: no guard, no run-time count.
    fn summary(&self) -> &str {
        self.reads.set(self.reads.get() + 1);
        self.summary.get_or_init(|| {
            println!("  (computing the summary)");
            let first: Vec<&str> = self.text.split_whitespace().take(3).collect();
            first.join(" ")
        })
    }
}
```

`reads` is updated in `summary(&self)` with `get` and `set`: no guard, no run-time check, no possible panic. The price is the restricted API: you cannot call a method on the value *inside* the cell, only move whole values in and out.

`Cell<&'a T>` and invariance

A `Cell` can hold a borrowed value, which makes its lifetime parameter part of the picture. This cursor remembers the last token it saw, through `&self`:

```
/// A `Cell` can hold a borrowed value too: `Cell<&'src str>` is fine as long
/// as every slice stored in it comes from the same `'src` text.
struct Cursor<'src> {
    last_token: Cell<&'src str>,
}

impl<'src> Cursor<'src> {
    fn remember(&self, token: &'src str) {
        self.last_token.set(token);
    }
}
```

This works because every token comes from the same source text, `'src`. Chapter 4 showed why `Cell<T>` is *invariant* in `T`, and here is the consequence in a setting of this chapter. The compiler rejects this program (`code/compile_fail/ch12_cell_invariance.rs`):

```
fn remember<'a>(slot: &Cell<&'a str>, token: &'a str) {
    slot.set(token);
}

pub fn caller(slot: &Cell<&'static str>) {
    let local = String::from("short-lived");
    remember(slot, &local); // would leave a dangling `&str` in `slot`
}
```

```
error[E0597]: `local` does not live long enough
  --> compile_fail/ch12_cell_invariance.rs:14:20
```

```
   |
13 |     let local = String::from("short-lived");
   |         ----- binding `local` declared here
14 |     remember(slot, &local); // would leave a dangling `&str` in `slot`
   |     ----------------^^^^^^_
   |     |               |
   |     |               borrowed value does not live long enough
   |     argument requires that `local` is borrowed for `'static`
15 | }
   | - `local` dropped here while still borrowed
```

`remember` asks for a slot and a token with the same `'a`. For a plain `&&'static str`, the compiler could shrink the slot's type to `&&'a str` (covariance), pick `'a` as short as `local`, and accept the call. For a `Cell`, it may not: `slot` would then let `remember` store a short-lived `&str` in a cell that the caller still believes to hold `&'static str`, and the caller could read it after `local` is gone. Since the cell's type cannot change, `'a` must be `'static`, and the label says so: "argument requires that `local` is borrowed for `'static`". Invariance is how the type system protects every type that can be written through a shared reference.

`OnceCell`: plain references, no guard

`OnceCell<T>` is a cell that can be written once. Its key method has a signature that may surprise you after the `RefCell` section:

```
impl<T> OnceCell<T> {
    pub fn get_or_init<F>(&self, f: F) -> &T
    where
        F: FnOnce() -> T;
}
```

It returns a plain `&T` with the lifetime of `&self`: no guard. The `summary` method of `Document` above uses it to compute the summary on first use and to return a `&str` that borrows the document:

```
let doc = Document::new("lifetimes are regions of the program");
let first = doc.summary();
let second = doc.summary(); // both references are alive at once
println!("{first:?} == {second:?}");
println!("summary read {} times", doc.reads.get());
```

```
  (computing the summary)
"lifetimes are regions" == "lifetimes are regions"
summary read 2 times
```

The summary is computed once, and two references to it are alive at the same time as the `Cell` counter is updated. This is sound because of what `OnceCell` does *not* offer: once the cell is full, nothing that takes `&self` changes the contents. A second `get_or_init` returns the existing value. The methods that can change or remove it (`take`, `get_mut`, `into_inner`) need `&mut self` or ownership, and those conflict with every outstanding `&T` in the ordinary, compile-time way. This program is rejected (`code/compile_fail/ch12_oncecell_take.rs`):

```
pub fn reset(summary: &mut OnceCell<String>) {
    let current = summary.get_or_init(|| String::from("draft"));
    summary.take(); // would free the `String` that `current` points to
    println!("{current}");
}
```

```
error[E0502]: cannot borrow `*summary` as mutable because it is also borrowed as immutable
  --> compile_fail/ch12_oncecell_take.rs:11:5
   |
10 |     let current = summary.get_or_init(|| String::from("draft"));
   |                   ------- immutable borrow occurs here
11 |     summary.take(); // would free the `String` that `current` points to
   |     ^^^^^^^^^^^^^ mutable borrow occurs here
12 |     println!("{current}");
   |                ------- immutable borrow later used here
```

So `OnceCell` needs only one bit of run-time state (empty or full). The aliasing rule for the contents is left to the borrow checker, which can enforce it because the dangerous operations require `&mut`. That is the design to aim for when you build your own types: the more of the aliasing rule you express with `&` and `&mut` in signatures, the less you have to check at run time.

12.6 `thread_local!` and closure-scoped borrows

A `thread_local!` declares a value of which each thread has its own copy. The declaration creates a `LocalKey<T>`, and the value is reached through methods of the key (`code/examples/ch12_thread_local.rs`):

```
thread_local! {
    static LOG: RefCell<Vec<String>> = const { RefCell::new(Vec::new()) };
    static DEPTH: Cell<u32> = const { Cell::new(0) };
}
```

```
fn log(line: &str) {
    LOG.with(|log| log.borrow_mut().push(line.to_owned()));
}

fn log_len() -> usize {
    LOG.with_borrow(|log| log.len()) // the result is a `usize`: nothing borrowed
}

fn first_entry() -> Option<String> {
    LOG.with_borrow(|log| log.first().cloned()) // an owned copy leaves the closure
}
```

```
log("main: start");
DEPTH.set(DEPTH.get() + 1);
thread::scope(|s| {
    s.spawn(|| {
        log("worker: start");
        println!("worker: log length {}, depth {}", log_len(), DEPTH.get());
    });
});
log("main: done");
println!("main: log length {}, depth {}", log_len(), DEPTH.get());
println!("main: first entry {:?}", first_entry());
```

```
worker: log length 1, depth 0
main: log length 2, depth 1
main: first entry Some("main: start")
```

The worker thread sees only its own entry and its own `DEPTH`, which is still 0; the main thread sees its two entries. `with_borrow` and `with_borrow_mut` (for `LocalKey<RefCell<T>>`) and `get` and `set` (for `LocalKey<Cell<T>>`) are shortcuts, stable since Rust 1.73.

Why `with` takes a closure

Every access goes through a closure. The signature of `LocalKey::with`, verbatim from the standard library, is:

```
pub fn with<F, R>(&'static self, f: F) -> R
where
    F: FnOnce(&T) -> R,
```

The key is `&'static`, but the value is not: each thread has its own, and it is destroyed when that thread exits. A `&'static T` to it could outlive its thread. So `with` lends the reference to `f` instead. `FnOnce(&T) -> R` elides the lifetime of `&T`, which makes the bound higher-ranked (Chapter 9): `for<'a> FnOnce(&'a T) -> R`. `with` picks `'a` inside its body, and `R` is fixed by the caller before that, so `R` cannot contain `'a`. Trying to return the reference anyway is rejected (`code/compile_fail/ch12_thread_local_escape.rs`):

```
thread_local! {
    static GREETING: String = String::from("hello");
}

pub fn greeting() -> &'static str {
    GREETING.with(|text| text.as_str())
}
```

```
error: lifetime may not live long enough
  --> compile_fail/ch12_thread_local_escape.rs:11:26
   |
11 |     GREETING.with(|text| text.as_str())
   |                    -----  ^^^^^^^^^^^^^ returning this value requires that `'1` must
outlive `'2`
   |                    |   |
   |                    |   return type of closure is &'2 str
   |                    has type `&'1 String`
```

This error has no code; it is the "lifetime may not live long enough" of closures. The labels name two anonymous lifetimes: `'1` is the lifetime of the closure's parameter `text` ("has type `&'1 String`"), and `'2` is the lifetime in the closure's return type ("return type of closure is `&'2 str`"). Returning `text`'s contents requires `'1` to outlive `'2`. But `'1` is chosen anew for each call, inside `with`, and `'2` is part of the type `R`, chosen outside. Nothing can make one outlive the other, which is the point: the borrow is confined to the closure.

Even inside the closure, `with` makes no promise for the time while the thread is being torn down: its documentation says that it "will `panic!()` if the key currently has its destructor running, and it **may** panic if the destructor has previously been run for this thread". The fallible variant `try_with` returns an error instead.

Lending instead of returning

`LocalKey::with` is one instance of a pattern you have now seen three times: `History::with_last` in this chapter, `thread::scope` in Chapter 7, and `LocalKey::with`. When the validity of a reference depends on something that must happen *after* its last use (a guard released, a thread joined, a thread-local torn down), the API cannot return the reference, because the caller could keep it past that point. It lends the reference to a closure, and a higher-ranked bound guarantees that the closure gives it back.

The cost is a less direct calling style; the gain is that misuse is a compile error instead of a panic, a deadlock or a dangling pointer.

12.7 Common confusions

"**`RefCell` turns off the borrow checker.**" It moves one check to run time: the conflict between guards of the same cell. Every guard is still an ordinary borrow of the cell, and the cell is still subject to every compile-time rule (`ch12_guard_outlives_cell.rs`).

"**A `Ref<'b, T>` can give me a `&'b T`.**" It gives you a reference tied to the guard *value*. That is what keeps the run-time count honest, and it is why getters return `Ref<'_, T>`, take a closure, or return an owned copy.

"**`match cell.borrow().len()` releases the guard, since `len()` returned a number.**" The guard is a temporary of the scrutinee and lives until the end of the `match`, whatever the type of the scrutinee's value. The same holds for `while let`; `if let` releases it before `else` in edition 2024.

"**`let _ = mutex.lock()` holds the lock.**" It releases it immediately, and rustc rejects it for that reason. `let _guard = mutex.lock().unwrap();` holds it.

"**A `MutexGuard<'static, T>` holds the lock forever.**" It holds the lock until it is dropped. The `'static` is the lifetime of the borrow of the mutex, possible because the mutex is a `static`.

"**`Cell` is a cheaper `RefCell`.**" It is a different contract: no references to the contents at all. When you need to call a method on the contents in place, you need `RefCell` (or a lock); when you only move values in and out, `Cell` is simpler and cannot panic.

12.8 Summary

- Interior mutability allows mutation through `&T`. It keeps the readers-or-writer rule, enforced at run time (`RefCell`, `Mutex`, `RwLock`) or by an API that cannot violate it (`Cell`, `OnceCell`). It does not remove lifetimes.
- `RefCell::borrow(&self) -> Ref<'_, T>`: the guard is a compile-time borrow of the cell and a run-time borrow of its contents. Conflicts between guards panic ("RefCell already borrowed", "RefCell already mutably borrowed"); conflicts with the cell itself are compile errors.
- References obtained through a guard live no longer than the guard. A getter cannot return `&T` from a `RefCell` (E0515); return a guard narrowed with `Ref::map` or `Ref::filter_map`, lend the data to a closure, or return an owned copy.
- A guard that is not bound to a variable is a temporary: it lives to the end of a `match`, of each `while let` iteration, or of the block when extended; in edition 2024 an `if let` releases it before `else`, and a tail expression before the block's locals.
- `MutexGuard` and the `RwLock` guards follow the same rules; a conflict waits instead of panicking, and on the same thread never returns. Keep critical sections short with statement-level temporaries, blocks or `drop`; never `let _ =` a guard. Guards of a `static` lock are `'static`.

- `Cell` never lends references, so it needs no guard; `Cell<T>` is invariant in `T`. `OnceCell::get_or_init` returns `&T` tied to `&self` because the contents never change through `&self` once set.
- `LocalKey::with` lends `&T` to a closure with a higher-ranked bound, so the reference cannot escape the call. Lending instead of returning is the general pattern when a reference must not outlive a cleanup step.

12.9 Exercises

1. Add to `History` (from `ch12_ref_map.rs`) a method `fn longest(&self) -> Option<Ref<'_, str>>` that returns the longest entry. Then write a caller that holds the result in a variable while calling `history.entries.borrow_mut()`, and observe the panic. Change the caller, not the method, so that it works. *Hint:* `Ref::filter_map` with `iter().max_by_key(..)`; the caller needs a shorter scope for the guard.
2. A work queue is a `RefCell<Vec<String>>`. Write a loop that pops jobs and, for every job that ends with `!`, pushes a follow-up job without the `!`. Write it first with `while let Some(job) = queue.borrow_mut().pop()` and run it; then fix it. *Hint:* the guard of a `while let` scrutinee lives for the whole body; pop in a `let` statement inside a `loop`.
3. Rewrite `History` with a `Mutex<Vec<String>>` so that it can be shared by threads in a `thread::scope`. Which of `entries`, `last`, `with_last` and `last_owned` can you keep with the same shape, and what must change? *Hint:* there is no stable `MutexGuard::map`; a method can still return the whole `MutexGuard<'_, Vec<String>>`.
4. Declare a thread-local `RefCell<Vec<String>>` of configuration lines and write `fn with_config<R>(f: impl FnOnce(&[String]) -> R) -> R`. Then try to write `fn config() -> &'static [String]` on top of it, read the error, and explain in one sentence which lifetime is chosen by whom. *Hint:* compare with `ch12_thread_local_escape.rs`; the closure's parameter lifetime is chosen inside `with`.
5. Give `Document` a field `words: OnceCell<Vec<String>>` filled on first use, and a method `fn words(&self) -> &[String]`. Explain why this needs no guard, then explain why `fn reset(&self)` that empties the cell cannot be written, while `fn reset(&mut self)` can (with `OnceCell` alone). *Hint:* look at which `OnceCell` methods take `&self` and which take `&mut self`.